

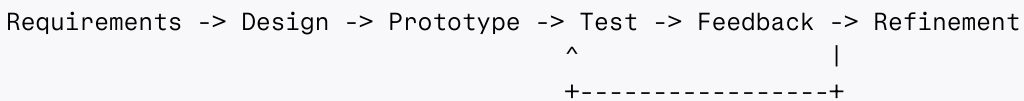
SDLC Section for the Distributed File Storage Project

1. Selected SDLC model

Iterative Prototyping Model

The proposed project will use the **Iterative Prototyping Model** of the Software Development Life Cycle. This model is appropriate because the system contains distributed components whose requirements may become clearer after an initial working prototype is demonstrated.

The team will follow this cycle:



An SDLC is a structured process for planning, designing, developing, testing, deploying, and maintaining software. [1] The prototyping approach is suitable for this project because the team can first implement basic chunking and reconstruction, demonstrate it to the supervisor, and then add deduplication, replication, and failure recovery based on feedback.

2. Why this model is suitable

The Iterative Prototyping Model is selected for the following reasons:

- The project has both clearly defined requirements and technically uncertain features.
- Distributed-node failure and recovery are easier to validate through a working prototype.
- The supervisor can review the system after each iteration.
- Errors in the architecture can be found before the complete system is developed.
- Features can be prioritized according to semester time constraints.
- The team can demonstrate measurable progress at every stage.

The team should not attempt erasure coding before the replicated MVP is stable. It should remain an optional advanced iteration.

3. SDLC phases and project deliverables

SDLC phase	Activities for this project	Deliverables
1. Planning	Define problem, aim, scope, team, schedule, and resources	Project charter and initial plan
2. Feasibility analysis	Evaluate technical, economic, operational, schedule, storage, and ethical feasibility	Feasibility study

SDLC phase	Activities for this project	Deliverables
3. Requirements analysis	Identify users, functional requirements, non-functional requirements, and constraints	SRS document and use-case list
4. System design	Design architecture, database, APIs, workflows, and security controls	Architecture diagram, ER diagram, API specification
5. Prototype development	Implement chunking, hashing, one node, and reconstruction	Prototype 1
6. Iteration 1	Add metadata server and multiple storage nodes	Prototype 2 and integration report
7. Iteration 2	Add deduplication and replication factor two	Prototype 3 and storage comparison
8. Iteration 3	Add heartbeat monitoring, failure simulation, and recovery	Fault-tolerant prototype
9. Testing	Perform functional, integration, performance, integrity, and failure tests	Test cases and test report
10. Deployment	Run the final prototype on a local cluster or virtual machines	Demonstration build and user guide
11. Maintenance and evaluation	Fix defects, improve documentation, and identify future extensions	Final report and future-work plan

4. Phase-by-phase explanation

Phase 1: Planning

The team defines the project title, problem statement, objectives, scope, technology stack, team responsibilities, and schedule.

Output: project charter, work-allocation table, initial schedule.

Phase 2: Feasibility analysis

The team determines whether the project can be completed with available skills, hardware, software, time, and storage. The result is that a three-node prototype with replication factor two is feasible.

Output: approved feasibility study.

Phase 3: Requirements analysis

The team identifies what the system must do.

Core functional requirements:

- Register and authenticate users.
- Upload files.
- Split files into chunks.
- Hash chunks.

- Distribute chunks.
- Reuse duplicate chunks.
- Replicate chunks.
- Download and reconstruct files.
- Detect corruption.
- Detect failed nodes.
- Recover missing replicas.
- Delete files safely.

Core non-functional requirements:

- Correct reconstruction.
- Integrity verification.
- Reasonable upload and download time.
- Secure file access.
- Extensibility for additional nodes.

Output: Software Requirements Specification.

Phase 4: System design

The team creates the architecture diagram, ER diagram, database schema, data-flow diagram, API list, sequence diagrams, and security plan.

Output: Software Design Document.

Phase 5: Prototype development

The first prototype should be deliberately small:

```
Local file -> split into chunks -> hash -> join chunks
```

At this stage, do not implement replication or erasure coding. The team must first prove that the original file can be reconstructed exactly.

Acceptance criterion:

```
SHA256(original file) == SHA256(reconstructed file)
```

Phase 6: Iteration 1—distributed placement

Create one metadata server and three storage-node services. Send chunks to different nodes and store their locations in the database.

Acceptance criteria:

- At least three nodes can run independently.
- Chunks are distributed across nodes.
- Metadata records correct sequence numbers and locations.
- Download reconstructs the correct file.

Phase 7: Iteration 2—deduplication and replication

Use the SHA-256 chunk hash as the content address. If the hash already exists, create a new reference instead of storing a duplicate. Store each unique chunk on two different nodes.

Acceptance criteria:

- Uploading the same file twice does not double physical chunk storage.
- Every active chunk has two locations when two healthy nodes are available.
- A reference count is updated correctly after upload and deletion.

Phase 8: Iteration 3—failure recovery

Add `/health` endpoints and periodic heartbeat requests. Stop a node during demonstration and show that a replica can still serve the file. Then create a replacement copy on a healthy node.

Acceptance criteria:

- Failed nodes are marked unavailable.
- The client tries a replica when the primary node fails.
- The system restores the replication factor after recovery.
- The reconstructed file passes the final hash check.

Phase 9: Testing

Testing must be performed at four levels:

Testing level	Example
Unit testing	Test chunk splitting and SHA-256 functions
Integration testing	Test client, metadata server, and storage node together
System testing	Test complete upload and download
Fault testing	Stop a node and verify replica recovery

Phase 10: Deployment

For the final class demonstration, run:

```
Metadata server: localhost:8000
Storage node 1: localhost:8001
Storage node 2: localhost:8002
Storage node 3: localhost:8003
Client: GUI or command line
```

Each node must use a separate storage directory. The team should prepare a script that starts all services consistently.

Phase 11: Maintenance and evaluation

After the demonstration, the team will fix defects, improve error messages, record known limitations, and propose future work such as erasure coding, encryption, a replicated metadata server, and content-defined chunking.

5. Iteration plan

Iteration	Main feature	Demonstration
Iteration 0	Chunking and reconstruction	Upload and reconstruct locally
Iteration 1	Metadata server and three nodes	Show chunks on different nodes
Iteration 2	Deduplication	Upload the same file twice and compare storage
Iteration 3	Replication	Show two locations for each unique chunk
Iteration 4	Failure recovery	Stop a node and download from a replica
Iteration 5	Final testing and UI	Present complete working system

6. SDLC acceptance criteria

The project will be considered successful when:

1. A user can upload a file.
2. The file is divided into multiple chunks.
3. Chunks are distributed across at least three storage-node processes.
4. Metadata records chunk order and locations.
5. The original file is reconstructed successfully.
6. Corruption is detected using SHA-256.
7. Duplicate chunks are not stored repeatedly.
8. A replica is available after one node fails.
9. The system can recover a missing replica.

10. The team can present actual test measurements.

7. SDLC risks and controls

Risk	SDLC control
Requirements change	Review requirements at the end of every iteration
Integration failure	Integrate small modules continuously
Erasur coding takes too long	Keep it as an optional extension
Database inconsistency	Use transactions and integration tests
Node failure bugs	Perform planned fault-injection tests
Unclear team responsibilities	Use a responsibility matrix and Git branches
Incomplete report	Maintain documentation during every phase

8. SDLC conclusion

The Iterative Prototyping SDLC model provides a controlled and practical development process for this project. It allows the team to build a small verified prototype first and progressively introduce distributed placement, deduplication, replication, monitoring, and recovery. This approach reduces technical risk, provides regular demonstrations for supervisor feedback, and ensures that a functioning MVP is completed before optional advanced features are attempted.

Reference

[1] IBM, “What is the Software Development Life Cycle (SDLC)?,” 2025.
<https://www.ibm.com/think/topics/sdlc>